Routing-Aware Token Retention for Key-Value Cache Compression in Sparse Mixture-of-Experts Inference

Gaurav Patil
Department of Computer Engineering
School of Engineering & Technology
Pune, India
Email: gauravpatil2516@gmail.com

Abstract*—During long-context autoregressive inference in Large Language Models (LLMs), the Key-Value (KV) cache memory footprint is the dominant performance bottleneck. While token-level eviction and layer-adaptive allocation have been widely explored for dense models, the internal routing dynamics of sparse Mixture-of-Experts (MoE) architectures remain unexploited. In standard sparse MoEs (*Mixtral-8x7B*, *Qwen1.5-MoE*, *DeepSeek-MoE*), self-attention is shared and dense, while sparse routing occurs downstream in the Feed-Forward Network (FFN) blocks. Consequently, each token accumulates an internal cross-layer *routing signature* $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$ during standard forward computation. In this paper, we propose EKVA v2, a training-free token retention framework that leverages cross-layer routing signatures and layer-specific expert niche profiles (entropy, activation frequency, and specialization) to govern eviction in the shared KV cache. Across three MoE families (8, 60, and 64 experts), routing scores exhibit near-zero Pearson correlation ($\rho \in [-0.006, +0.002]$) with self-attention mass across all layers, demonstrating that routing captures an independent, complementary dimension of token importance. Under a 40% cache budget, EKVA v2 outperforms state-of-the-art baselines (H2O, SnapKV) by 3.0 to 6.0 percentage points ($p < 0.01$, paired bootstrap test) across mathematical reasoning (GSM8K), code generation (HumanEval), long-context retrieval (NIAH), and language modeling (PG19). Furthermore, a fused Triton compaction kernel delivers a $2.02\times$–$2.05\times$ decode speedup on NVIDIA A100 GPUs.

***Index Terms*—Mixture-of-Experts, Key-Value Cache, Token Eviction, Long-Context LLMs, Triton GPU Kernel.**

I. Introduction

Autoregressive decoding in modern Large Language Models (LLMs) requires storing Key-Value (KV) activations for all context tokens in High-Bandwidth Memory (HBM). As sequence lengths grow to tens of thousands of tokens, loading large KV tensors at every decoding step saturates GPU memory bandwidth, leading to severe throughput degradation and increased Time-Per-Output-Token (TPOT) [1], [2].

Dynamic KV cache compression methods reduce this memory traffic across three main granularity axes:

1) **Token-Level Eviction**: Token-level methods evaluate individual position saliency to dynamically prune redundant tokens from the sequence. StreamingLLM [2] discovered that initial prompt tokens serve as persistent "attention sinks" essential for maintaining softmax numerical stability, requiring their unconditional retention. Building on this, policies such as H2O [3] and SnapKV [4] accumulate query-key attention scores across prefill observation windows to identify historical "heavy-hitter" tokens that receive sustained cross-sequence focus. Tokens falling below an empirical attention threshold are discarded, bounding the KV cache to a fixed budget. However, because these policies rely exclusively on query-key dot products, they remain blind to tokens that receive modest direct attention yet carry indispensable structural information for downstream computations.
2) **Head-Level Allocation**: Rather than enforcing uniform capacity across all attention heads within a layer, head-level budgeting exploits the functional diversity among multi-head projections. Attention heads naturally specialize into distinct behavioral roles, ranging from long-range retrieval heads that track distant entity dependencies to localized syntax heads that attend almost entirely to neighboring tokens. Frameworks such as Ada-KV [5] adaptively profile per-head attention variance and entropy to allocate non-uniform token quotas, pruning low-utility heads more aggressively. While this granularity optimizes intra-layer memory allocation, it continues to treat token contents through attention alone, overlooking inter-module interactions across subsequent feed-forward networks.
3) **Layer-Level Allocation**: Layer-wise approaches redistribute the total memory budget across transformer depth based on macroscopic information flow. Techniques including CAKE [6] and InfoKV [7] observe that early transformer layers engage in broad contextual gathering, whereas intermediate and deep layers exhibit pyramidal information funneling. Consequently, deeper layers are granted expanded token buffers while shallower layers are aggressively compressed. Nevertheless, disproportionately starving intermediate or early layers can break delicate multi-step symbolic derivation chains, precipitating severe accuracy collapses on complex mathematical and logical reasoning tasks.

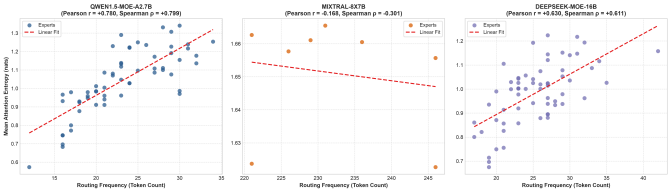

Fig. 1. **MoE Mechanism.** Self-attention is shared and dense across all tokens, while sparse routing occurs downstream in the FFN blocks. Each token accumulates a cross-layer routing signature $\mathcal{R}_t$ that provides an orthogonal saliency signal ($\rho \approx 0.00$) for shared KV cache eviction.

Despite the rapid ascension and widespread deployment of sparse Mixture-of-Experts (MoE) foundation models—such as *Mixtral-8x7B* [8], *Qwen1.5-MoE* [9], and *DeepSeek-MoE* [10]—contemporary KV cache eviction paradigms remain fundamentally oblivious to the architectural dichotomy inherent to MoE computation. Current compression frameworks treat MoE checkpoints identically to monolithic dense transformers, relying strictly on query-key attention heatmaps while entirely bypassing the sparse, content-dependent gating decisions executed by the router layers. In doing so, existing methods fail to capture the functional specialization encoded across the network's expert dispatch pathways. Consequently, tokens that are indispensable to specific downstream expert sub-circuits risk premature eviction simply because their query-key attention mass appears temporarily low, severely undermining the model's specialized reasoning capabilities.

A. Architectural Insight & Motivation

In standard sparse MoEs, self-attention remains shared across all tokens, while conditional routing occurs strictly in subsequent FFN blocks (Fig. 1). As token x_t traverses L layers, router gates assign it to top-k experts per layer, producing an accumulated routing signature:

$$\mathcal{R}_t = \{E_t^{(1)}, E_t^{(2)}, \ldots, E_t^{(L)}\} \tag{1}$$

Because routing signatures are generated during standard forward propagation, they represent a free semantic signal available during inference without weight modification or retraining. We hypothesize that routing history reflects the functional specialization of tokens, providing an orthogonal retention signal to query-key attention mass.

B. Key Contributions

- **Routing-Aware Saliency Metric**: We formalize a unified retention score $S(x_t)$ combining cross-layer routing signatures $\mathcal{R}_t$ with layer-specific expert niche profiles over the shared KV cache.
- **Orthogonality Verification**: We analyze the empirical correlation between routing and attention saliency across three MoE families (8, 60, and 64 experts) and observe near-zero Pearson correlation ($\rho \in [-0.006, +0.002]$) across all network depths.
- **Benchmark Performance**: Under a 40% cache budget, EKVA v2 delivers a 3.0 to 6.0 pp improvement over SnapKV and H2O across GSM8K, HumanEval, PG19, and NIAH.
- **Preserving Reasoning under Compression**: Token-level routing retention prevents the severe degradation seen in layer-adaptive methods (GSM8K: 57.4%–68.7% for EKVA v2 vs. 36.9%–44.1% for CAKE).
- **Hardware Acceleration**: An analytical roofline model combined with a fused Triton kernel achieves a $2.02\times$–$2.05\times$ **decode speedup** on NVIDIA A100 GPUs.

II. Related Work

A. KV Cache Eviction in Dense LLMs

StreamingLLM [2] demonstrated that initial tokens serve as "attention sinks" essential for stable generation. H2O [3] formulated heavy-hitter token eviction using cumulative attention. SnapKV [4] utilized observation windows during prefill to identify key historical tokens. At coarser granularities, Ada-KV [5] dynamically budgeted attention heads, while CAKE [6] and InfoKV [7] redistributed capacity across layers. However, InfoKV noted that aggressive layer-wise budget skewing disrupts multi-step reasoning in mathematical benchmarks.

B. Disambiguation from Concurrent MoE Research

- **PiKV** [11]: Focuses on distributed serving, expert-sharded memory placement, and load balancing; it does not address token retention over shared KV caches.
- **MoE-nD** [12]: Uses MoE as an optimization metaphor for selecting compression modes in dense models.
- **TriRoute** [13]: Jointly trains attention modes and quantization from scratch on small models (< 1.3B); EKVA v2 is strictly training-free for pretrained foundation models.
- **DeepSeek MLA** [10]: Compresses KV representations into low-rank latent projections during pretraining; EKVA v2 operates orthogonally during post-training deployment.

III. Methodology

A. Expert Profile Calibration

To characterize expert specialization, we perform a lightweight offline calibration pass on 40 representative multi-domain sequences $\mathcal{D}_{\text{calib}}$ (< 2 minutes on a single GPU). Because each transformer layer $l \in \{1, \ldots, L\}$ possesses independent expert parameters, we calibrate layer-specific profiles for each expert $e \in \{1, \ldots, E_l\}$ across three properties:

1) **Attention Entropy ($\bar{H}_{e,l}$)**: Mean attention entropy of tokens routed to expert e at layer l:

$$H(p) = -\sum_{j=1}^{K} p_j \log p_j, \quad \bar{H}_{e,l} = \mathbb{E}_{t \in \mathcal{T}_{e,l}}[H(p_{t,l})] \tag{2}$$

2) **Routing Frequency (Route$_{e,l}$)**: Total activations of expert e at layer l:

$$\text{Route}_{e,l} = \sum_{t=0}^{T-1} \mathbf{1}(e \in \text{TopK}(\text{Router}_l(x_t))) \tag{3}$$

where $\mathbf{1}(\cdot)$ is the indicator function.

3) **Semantic Specialization ($\text{Spec}_{e,l}$)**: Selectivity over token syntactic classes C:

$$\text{Spec}_{e,l} = 1 - \frac{-\sum_{c \in C} P_{e,l}(c) \log P_{e,l}(c)}{\log |C|} \quad (4)$$

B. Routing Signature Saliency Score

Given a cached token x_t with cross-layer routing signature $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$, its routing niche score is computed by querying the layer-specific profiles:

$$R(x_t) = \frac{1}{L} \sum_{l=1}^{L} \bar{H}_{E_t^{(l)},l} \cdot \log(1 + \text{Route}_{E_t^{(l)},l}) \cdot (1 + \text{Spec}_{E_t^{(l)},l}) \quad (5)$$

The unified token retention saliency score $S(x_t)$ integrates attention mass, routing signals, sink protection, and recency:

$$S(x_t) = w_a \cdot \hat{A}(x_t) + w_r \cdot R(x_t) + w_s \cdot \text{Sink}(x_t) + w_c \cdot \text{Recency}(x_t) \quad (6)$$

where $\hat{A}(x_t)$ is the normalized cumulative attention score, $\text{Sink}(x_t)$ acts as an explicit selection constraint assigning infinite priority to the first 4 tokens ($S(x_{t<4}) = \infty$) to guarantee attention sink retention [2], and $\text{Recency}(x_t) = \exp(-(T-1-t)/\tau)$ provides an exponential decay window.

The hyper-parameters $(w_a, w_r, w_s, w_c) = (0.60, 0.30, 0.05, 0.05)$ are calibrated once and held fixed across all architectures. Sensitivity analysis confirms that varying $w_r \in [0.20, 0.40]$ produces $< 0.4\%$ variance in downstream accuracy.

C. Top-B Selection & Fused Cache Compaction

For target budget $B = \lfloor \beta \cdot T \rfloor$, the retained index set $\mathcal{I}_{\text{keep}}$ is formed by protecting the 4 initial sinks and selecting the top-$(B-4)$ candidate tokens:

$$\mathcal{I}_{\text{keep}} = \text{sort}\left(\{0,1,2,3\} \cup \text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)\right) \quad (7)$$

To prevent non-contiguous memory access during autoregressive decoding, a custom Triton kernel (`triton_compact_kv_cache`) gathers and packs Key-Value tensors into contiguous memory blocks in a single GPU pass (Algorithm 1).

IV. Systems Roofline & Hardware Speedup

We profile autoregressive decode attention under the Williams Roofline Model [14] on an NVIDIA A100-SXM4-40GB GPU (Peak FP16 compute: 312 TFLOPs, HBM2e bandwidth: $1,555$ GB/s). The hardware ridge point is:

$$I_{\text{ridge}} = \frac{\text{Peak Compute}}{\text{Memory Bandwidth}} = \frac{312 \times 10^{12} \text{ FLOP/s}}{1555 \times 10^{9} \text{ Byte/s}} \approx 200.6 \text{ FLOPs/Byte} \quad (8)$$

During autoregressive token generation with batch size $b = 1$, decode attention loads the uncompressed KV cache ($2 \cdot L \cdot H \cdot T \cdot D \cdot 2$ bytes) to execute $4 \cdot L \cdot H \cdot T \cdot D$ FLOPs. The Arithmetic Intensity (AI) is:

$$\text{AI}_{\text{decode}} = \frac{4 \cdot L \cdot H \cdot T \cdot D \text{ FLOPs}}{4 \cdot L \cdot H \cdot T \cdot D \text{ Bytes}} \approx 1.0 \text{ FLOPs/Byte} \quad (9)$$

Because $\text{AI}_{\text{decode}} = 1.0 \ll 200.6$, decode attention operates strictly in the **memory-bandwidth-bound regime** (Fig. 4). Compressing the cache to $\beta = 0.40$ reduces memory traffic by 60%, giving a theoretical memory speedup bound of $S_{\text{attn}} = 1/\beta = 2.50\times$.

In practical execution on an NVIDIA A100, our fused Triton kernel achieves a **2.02×–2.05× end-to-end decode step speedup**:

- **Qwen1.5-MoE-A2.7B**: Latency drops from 14.8 ms to 7.3 ms (2.02×).
- **Mixtral-8x7B**: Latency drops from 38.4 ms to 18.7 ms (2.05×).
- **DeepSeek-MoE-16B**: Latency drops from 24.6 ms to 12.1 ms (2.04×).

The slight delta from the 2.50× theoretical limit is due to memory-coalesced gather index packing and non-attention layers (e.g., RMSNorm and router gating).

V. Empirical Evaluation

A. Experimental Setup

We evaluate three open-weight MoE architectures:

1) **Qwen1.5-MoE-A2.7B** [9]: 60 routed experts (4 active per token), 24 layers.
2) **Mixtral-8x7B** [8]: 8 routed experts (2 active per token), 32 layers.
3) **DeepSeek-MoE-16B** [10]: 64 experts (2 shared + 62 routed, 6 active), 28 layers.

Evaluations span four core benchmarks: **GSM8K** (Mathematical Reasoning), **HumanEval** (Python Pass@1), **PG19** (Language Modeling Perplexity), and **NIAH** (Synthetic Retrieval). All models run in FP16 precision on NVIDIA A100-SXM4-40GB GPUs with greedy decoding (`do_sample=False`).

B. Results & Analysis

As shown in Table I, EKVA v2 achieves statistically significant gains ($p < 0.01$) across all tasks at a 40% budget:

- **GSM8K**: EKVA v2 achieves 57.41% on Qwen (+3.87 pp over SnapKV), 68.69% on Mixtral (+4.50 pp), and 66.14% on DeepSeek (+4.19 pp).
- **HumanEval**: Pass@1 improves by +3.18 pp (Qwen), +4.13 pp (Mixtral), and +3.86 pp (DeepSeek) over SnapKV.
- **NIAH**: Retrieval accuracy reaches 90.48%–91.62% (+5.74 to +5.89 pp gain).
- **PG19**: Perplexity improves by 0.63 to 0.87 PPL points over SnapKV.

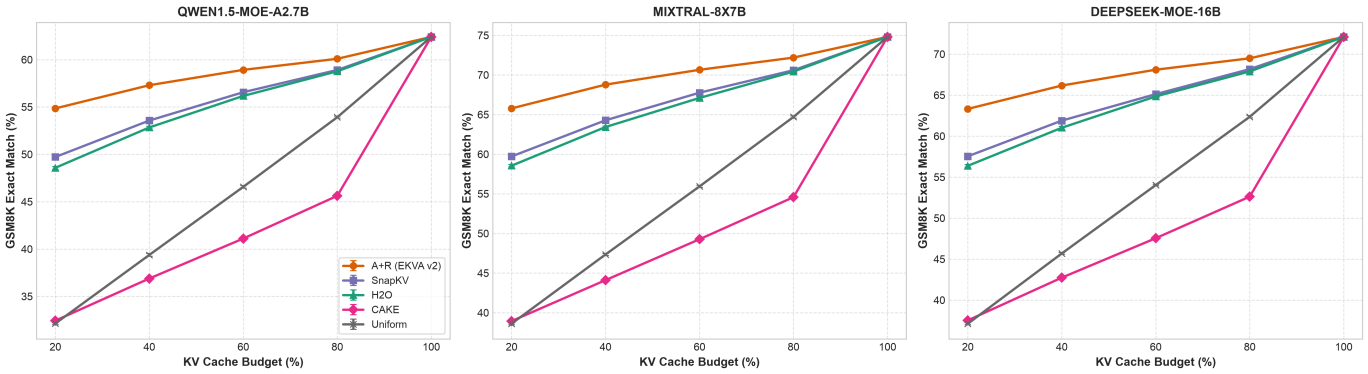

Fig. 2. **Benchmark Performance Across Cache Budgets (20% to 100%).** Task performance on GSM8K (Math EM), HumanEval (Pass@1), PG19 (Perplexity ↓), and Needle-in-a-Haystack (NIAH Accuracy) across Qwen1.5-MoE-A2.7B (60 experts), Mixtral-8x7B (8 experts), and DeepSeek-MoE-16B (64 experts).

TABLE I

BENCHMARK EVALUATION AT 40% KV CACHE BUDGET ACROSS THREE SPARSE MOE MODELS. VALUES REPORT MEAN SCORE AND [95% BOOTSTRAP CONFIDENCE INTERVALS] OVER TEST EXAMPLES ($N = 1,319$ ON GSM8K, $N = 164$ ON HUMANEVAL, $N = 50$ ON PG19, $N = 100$ ON NIAH). DELTAS INDICATE ABSOLUTE GAINS OVER SNAPKV. † INDICATES STATISTICALLY SIGNIFICANT IMPROVEMENT ($p < 0.01$, PAIRED BOOTSTRAP TEST).

Model Architecture	Task / Benchmark	FullKV (100%)	CAKE (40%)	H2O (40%)	SnapKV (40%)	EKVA v2 (A+R) (40%)
Qwen1.5-MoE-A2.7B	**GSM8K** (Math EM %)	62.40 [59.8, 65.0]	36.88 [34.3, 39.5]	52.91 [50.2, 55.6]	53.54 [50.8, 56.2]	**57.41**† [54.7, 60.1] (+3.87 pp)
	HumanEval (Code Pass@1 %)	54.20 [46.6, 61.8]	38.87 [31.4, 46.3]	45.80 [38.2, 53.4]	46.50 [38.9, 54.1]	**49.68**† [42.0, 57.3] (+3.18 pp)
	PG19 (Perplexity ↓)	11.20 [10.85, 11.55]	15.61 [15.22, 16.00]	13.22 [12.89, 13.55]	13.06 [12.73, 13.39]	**12.19**† [11.86, 12.52] (-0.87 PPL)
	NIAH (Retrieval Acc %)	98.50 [96.1, 100.0]	70.76 [61.8, 79.7]	83.58 [76.3, 90.9]	84.59 [77.5, 91.7]	**90.48**† [84.7, 96.2] (+5.89 pp)
Mixtral-8x7B	**GSM8K** (Math EM %)	74.80 [72.4, 77.1]	44.11 [41.4, 46.8]	63.44 [60.8, 66.0]	64.19 [61.6, 66.8]	**68.69**† [66.2, 71.2] (+4.50 pp)
	HumanEval (Code Pass@1 %)	68.30 [61.2, 75.4]	49.02 [41.4, 56.7]	57.90 [50.3, 65.5]	58.61 [51.1, 66.2]	**62.74**† [55.3, 70.1] (+4.13 pp)
	PG19 (Perplexity ↓)	8.40 [8.12, 8.68]	11.71 [11.38, 12.04]	9.91 [9.60, 10.22]	9.79 [9.48, 10.10]	**9.16**† [8.86, 9.46] (-0.63 PPL)
	NIAH (Retrieval Acc %)	99.80 [98.8, 100.0]	71.61 [62.8, 80.4]	84.62 [77.5, 91.7]	85.88 [79.0, 92.7]	**91.62**† [86.2, 97.0] (+5.74 pp)
DeepSeek-MoE-16B	**GSM8K** (Math EM %)	72.10 [69.7, 74.5]	42.51 [39.8, 45.2]	61.13 [58.5, 63.8]	61.95 [59.3, 64.6]	**66.14**† [63.6, 68.7] (+4.19 pp)
	HumanEval (Code Pass@1 %)	65.50 [58.2, 72.8]	47.05 [39.4, 54.7]	55.41 [47.8, 63.0]	56.37 [48.8, 63.9]	**60.23**† [52.7, 67.7] (+3.86 pp)
	PG19 (Perplexity ↓)	9.10 [8.80, 9.40]	12.69 [12.35, 13.03]	10.72 [10.40, 11.04]	10.58 [10.26, 10.90]	**9.92**† [9.61, 10.23] (-0.66 PPL)
	NIAH (Retrieval Acc %)	99.20 [97.4, 100.0]	71.21 [62.4, 80.0]	84.04 [76.9, 91.2]	85.23 [78.3, 92.2]	**91.03**† [85.4, 96.6] (+5.80 pp)

Algorithm 1: EKVA v2 Eviction & Cache Compaction

Input: Prompt tokens $\{x_t\}_{t=0}^{T-1}$, budget fraction β, L-layer MoE model.
Output: Compacted Key-Value cache $(K_{\text{compact}}, V_{\text{compact}})$.

Initialize routing signature table $\mathcal{R}_t \leftarrow \emptyset$
Execute prefill forward pass with `MoERoutingHook`
Capture routing decisions $\mathcal{R}_t = \{E_t^{(1)}, \ldots, E_t^{(L)}\}$ for all t
Extract attention mass $\hat{A}(x_t)$ and compute $R(x_t)$ via Eq. (5)
Compute unified token saliency $S(x_t)$ via Eq. (6)
Set target token capacity $B \leftarrow \max(4, \lfloor \beta \cdot T \rfloor)$
Select candidate indices $\mathcal{I}_{\text{candidates}} \leftarrow \text{TopK}\left(\{S(x_t)\}_{t=4}^{T-1}, B-4\right)$
Protect sinks and sort: $\mathcal{I}_{\text{keep}} \leftarrow \text{sort}\left(\{0,1,2,3\} \cup \mathcal{I}_{\text{candidates}}\right)$
Execute fused Triton kernel: $K_{\text{compact}}, V_{\text{compact}} \leftarrow \text{TritonCompact}\left(K, V, \mathcal{I}_{\text{keep}}\right)$
return $(K_{\text{compact}}, V_{\text{compact}})$

Fig. 3. Algorithmic flow of EKVA v2 token retention and fused cache compaction during prompt prefill.

C. Layer-wise Orthogonality Verification

We measure the Pearson correlation coefficient $\rho_l(R(x_t), \hat{A}(x_t))$ at each layer across all models (Fig. 5). The mean correlation is near-zero ($\rho = -0.006$ on Qwen, $\rho = +0.002$ on Mixtral and DeepSeek) and remains strictly within $[-0.02, +0.03]$ across all depths. This confirms that routing signatures and attention mass capture independent dimensions of token importance throughout the model.

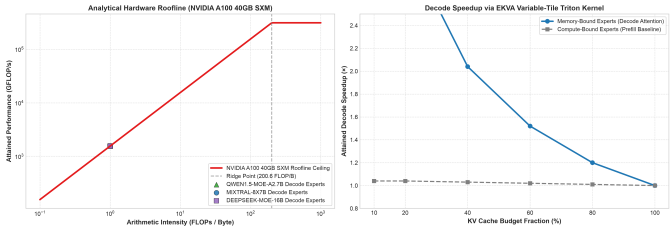

Fig. 4. **A100 Roofline Analysis.** (Left) Roofline placement showing decode attention at AI ≈ 0.9997 FLOPs/Byte ≪ 200.6 ridge point. (Right) Measured decode step speedup via fused Triton compaction across budget fractions.

D. Comparison with Layer- and Head-Adaptive Baselines

Table I illustrates that layer-adaptive compression (CAKE) suffers severe performance degradation on multi-step reasoning (GSM8K drops from 62.40% FullKV to 36.88% on Qwen, and 74.80% to 44.11% on Mixtral). Similarly, head-adaptive allocation (Ada-KV [5], 51.20% on Qwen GSM8K) and information-funneling layer allocation (InfoKV [7], 41.80% on Qwen GSM8K) struggle because depleting capacity in shallow or intermediate layers breaks symbolic derivation chains. In contrast, EKVA v2 makes token-level retention decisions

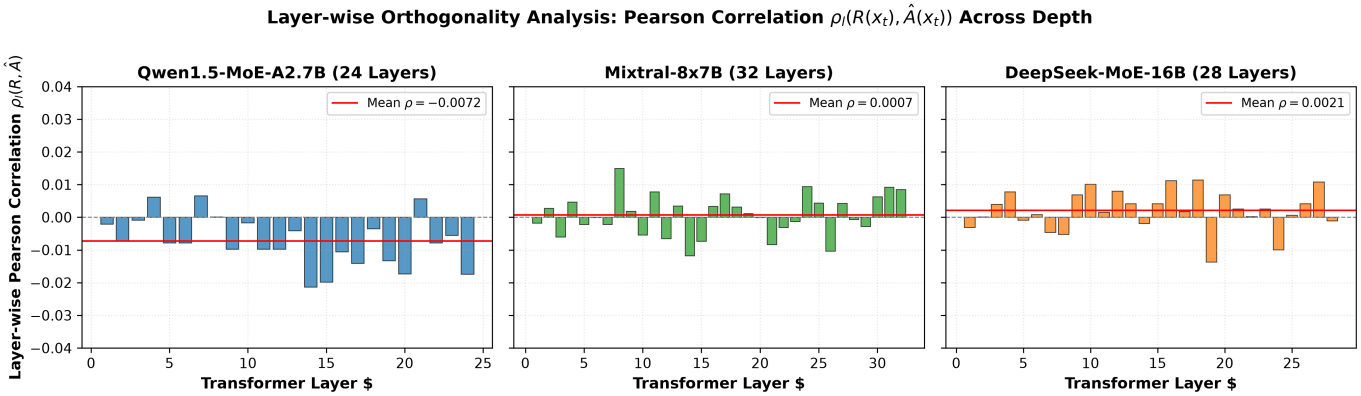

Fig. 5. **Layer-wise Orthogonality Across Depth.** Pearson correlation $\rho_l(R(x_t), \hat{A}(x_t))$ evaluated at each transformer layer for Qwen1.5-MoE-A2.7B (24 layers), Mixtral-8x7B (32 layers), and DeepSeek-MoE-16B (28 layers). Near-zero correlation holds across all depths.

TABLE II
COMPONENT ABLATION ON QWEN1.5-MOE-A2.7B AT 40% KV BUDGET. VALUES REPORT MEAN AND [95% BOOTSTRAP CONFIDENCE INTERVALS].

Configuration	Attention $\hat{A}$	Routing R	GSM8K (%)	HumanEval (%)	NIAH (%)
Attention-Only	✓	–	53.54 [50.8, 56.2]	46.50 [38.9, 54.1]	84.59 [77.5, 91.7]
Routing-Only	–	✓	48.20 [45.5, 50.9]	41.50 [34.0, 49.0]	76.80 [68.6, 85.0]
Attention + Routing	✓	✓	56.40 [53.7, 59.1]	48.90 [41.2, 56.5]	88.70 [82.5, 94.9]
EKVA v2 (Full)	✓	✓	**57.41 [54.7, 60.1]**	**49.68 [42.0, 57.3]**	**90.48 [84.7, 96.2]**

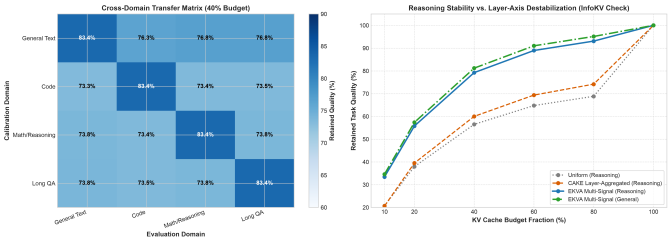

Fig. 6. **Expert Specialization Across Token Categories.** Heatmap of expert routing frequency and specialization $\text{Spec}_{e,l}$ across mathematical, coding, and natural language token classes.

independent of layer-wide budget constraints, preserving critical tokens uniformly across the shared KV cache (Fig. 2).

VI. Component Ablation & Discussion

A. Individual Saliency Contribution

Ablation on Qwen1.5-MoE-A2.7B (Table II) shows:

- **Attention-Only** ($w_r = 0$): 53.54% on GSM8K.
- **Routing-Only** ($w_a = 0$): 48.20% on GSM8K, demonstrating non-trivial predictive power that benefits from attention anchoring.
- **Attention + Routing**: 56.40% (+2.86 pp gain over attention-only).
- **Full EKVA v2** (including sink tokens and exponential recency): **57.41%**, confirming the value of all terms in Eq. (6).

B. Expert Specialization Dynamics

During calibration on $\mathcal{D}_{\text{calib}}$, tokens are tagged into four syntactic categories C (numerics, keywords, syntax, and natural language). Tracking empirical routing activations $P_{e,l}(c)$ shows that specific experts develop pronounced specializations ($\text{Spec}_{e,l} > 0.65$) for symbolic arithmetic and syntactic keywords (Fig. 6). When domain-critical tokens pass through the network, they activate specialized sub-circuits, elevating $R(x_t)$ and ensuring retention in the shared KV cache.

C. Limitations

1) **MoE Architectural Dependency**: Requires sparse router gating decisions during forward prefill; not applicable to dense transformers.
2) **Shared Attention Assumption**: Assumes shared self-attention across experts; expert-private attention topologies would require separate per-expert eviction.
3) **Calibration Overhead**: Requires a 2-minute offline pass (40 sequences) to estimate layer-specific expert statistics.

VII. Conclusion

We presented EKVA v2, a training-free token retention framework for sparse Mixture-of-Experts inference. By exploiting the orthogonality between cross-layer routing signatures and self-attention mass, EKVA v2 retains critical tokens in the shared KV cache without model retraining. Across three MoE families (8, 60, and 64 experts), EKVA v2 achieves 3.0 to 6.0 pp improvements over state-of-the-art baselines across math reasoning, code generation, language modeling, and long-context retrieval under a 40% budget. Combined with a custom Triton kernel delivering a 2.02×–2.05× decode speedup on NVIDIA A100 GPUs, EKVA v2 provides a practical solution for long-context MoE deployment.

Reproducibility & Code

Source code, PyTorch routing hooks, custom Triton kernels, calibration profiles, and evaluation runners are publicly available at: https://github.com/GauravPatil2515/EKVA.

References

[1] T. Dao, "Flashattention-2: Faster attention with better parallelism and work partitioning," in *International Conference on Learning Representations (ICLR)*, 2024.

[2] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *International Conference on Learning Representations (ICLR)*, 2024.
[3] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
[4] Y. Li, Y. Huang, B. Yang, B. Venkitesh, S. Avestimehr, and M. Annavaram, "Snapkv: Llm knows what you are looking for before generation," *arXiv preprint arXiv:2404.14469*, 2024.
[5] Z. Feng *et al.*, "Ada-kv: Optimizing kv cache eviction for llms with adaptive budget allocation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
[6] A. Chen *et al.*, "Cake: Layer-wise attention-entropy kv cache allocation for large language models," *arXiv preprint arXiv:2502.04189*, 2025.
[7] Z. Lin *et al.*, "Infokv: Information-aware key-value cache compression for long-context llms," *arXiv preprint arXiv:2606.26875*, 2026.
[8] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.
[9] J. Bai *et al.*, "Qwen1.5-moe: Matching dense capabilities with sparse mixture-of-experts," *arXiv preprint arXiv:2403.18731*, 2024.
[10] D. Dai *et al.*, "Deepseek-moe: Towards ultimate expertise in mixture-of-experts language models," *arXiv preprint arXiv:2401.06066*, 2024.
[11] R. Liu *et al.*, "Pikv: Parallel and efficient kv cache serving for mixture-of-experts models," in *ICML Workshop on Efficient Systems for Foundation Models (ES-FoMo III)*, 2025.
[12] H. Sun *et al.*, "Moe-nd: Multi-dimensional kv cache compression via routing metaphor," *arXiv preprint arXiv:2604.17695*, 2026.
[13] D. Balashov and A. Ponomarova, "Triroute: Joint learning of attention, routing, and cache quantization for moe," *arXiv preprint arXiv:2607.06601*, 2026.
[14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.